Notebook

September 30, 2026

1 Exercício: Random Forest e AdaBoost

Use `load_breast_cancer()` do scikit-learn. Neste conjunto, `0 = maligno` e `1 = benigno`. Adote `random_state=42`. Entregue o notebook com código executado e respostas fundamentadas nos resultados. Trata-se de dados didáticos; a comparação não constitui validação clínica.

1.1 Questão 1 — Conheça e prepare os dados

- Carregue o conjunto em um `DataFrame` com os 30 atributos e a variável alvo. Informe dimensões, tipos, valores ausentes, linhas duplicadas e quantidade por classe.
- Descreva o tratamento necessário (ou explique por que nenhum é necessário). Separe treino (60%), validação (20%) e teste (20%) de modo estratificado; mantenha o teste reservado para a questão 5. Por que preservar a proporção das classes?

[1]:
```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name='target')
df = pd.concat([X, y], axis=1)

print("Dimensoes:", df.shape)
print("Tipos de dados:\n", df.dtypes.value_counts())
print("Valores ausentes:", df.isnull().sum().sum())
print("Linhas duplicadas:", df.duplicated().sum())
print("Quantidade por classe:\n", df['target'].value_counts())

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.4, random_state=42, stratify=y
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp
)

print(f"\nDivisao: Treino={len(X_train)}, Validacao={len(X_val)},␣
 ↪Teste={len(X_test)}")
```

```
Dimensoes: (569, 31)
Tipos de dados:
 float64    30
int64       1
Name: count, dtype: int64
Valores ausentes: 0
Linhas duplicadas: 0
Quantidade por classe:
 target
1    357
0    212
Name: count, dtype: int64

Divisao: Treino=341, Validacao=114, Teste=114
```

Resposta:

- Dimensoes: 569 linhas e 31 colunas (30 atributos continuos float64 e a variavel alvo inteira int64).
- Valores ausentes: 0.
- Linhas duplicadas: 0.
- Quantidade por classe: 212 malignos (0) e 357 benignos (1).
- Tratamento necessario: nenhum tratamento adicional e necessario, pois nao existem dados faltantes e os algoritmos baseados em arvores nao exigem normalizacao ou padronizacao de escala.
- Preservar a proporcao das classes: a estratificacao garante que treino, validacao e teste tenham a mesma distribuicao original (cerca de 37.3% malignos e 62.7% benignos), evitando que subconjuntos fiquem com poucas amostras da classe minoritaria e gerem avaliacoes enviesadas.

1.2 Questão 2 — Random Forest

- Treine um `RandomForestClassifier(n_estimators=100, random_state=42)`. Na validação, apresente matriz de confusão com ordem `[0, 1]`, accuracy, precision, recall e F1 para **maligno** (`pos_label=0`).
- Teste pelo menos quatro combinações de `n_estimators` (por exemplo, 50 e 200) e `max_depth` (por exemplo, 3 e `None`). Compare os resultados de validação e explique o efeito dos parâmetros.

[2]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
 ↪f1_score, confusion_matrix

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_val = rf.predict(X_val)

print("Matriz de Confusao (ordem [0, 1]):\n", confusion_matrix(y_val,␣
 ↪y_pred_val, labels=[0, 1]))
```

```
print(f"Accuracy: {accuracy_score(y_val, y_pred_val):.4f}")
print(f"Precision (0): {precision_score(y_val, y_pred_val, pos_label=0):.4f}")
print(f"Recall (0): {recall_score(y_val, y_pred_val, pos_label=0):.4f}")
print(f"F1 (0): {f1_score(y_val, y_pred_val, pos_label=0):.4f}")

combos = [(50, 3), (50, None), (200, 3), (200, None)]
rf_results = []

for n_est, depth in combos:
    m = RandomForestClassifier(n_estimators=n_est, max_depth=depth,␣
 ↪random_state=42)
    m.fit(X_train, y_train)
    p = m.predict(X_val)
    rf_results.append({
        'n_estimators': n_est,
        'max_depth': str(depth),
        'Accuracy': accuracy_score(y_val, p),
        'Precision': precision_score(y_val, p, pos_label=0),
        'Recall': recall_score(y_val, p, pos_label=0),
        'F1': f1_score(y_val, p, pos_label=0)
    })

pd.DataFrame(rf_results)
```

```
Matriz de Confusao (ordem [0, 1]):
 [[39  3]
 [ 1 71]]
Accuracy: 0.9649
Precision (0): 0.9750
Recall (0): 0.9286
F1 (0): 0.9512
```

[2]:

	n_estimators	max_depth	Accuracy	Precision	Recall	F1
0	50	3	0.964912	0.975	0.928571	0.95122
1	50	None	0.964912	0.975	0.928571	0.95122
2	200	3	0.964912	0.975	0.928571	0.95122
3	200	None	0.964912	0.975	0.928571	0.95122

Resposta:

- Resultados na validacao do modelo inicial: Matriz de confusao [[39, 3], [1, 71]], Acuracia = 0.9649, Precisao = 0.9750, Recall = 0.9286 e F1 = 0.9512.
- Efeito dos parametros:
  - n_estimators: numero de arvores na floresta. Aumentar esse valor reduz a variancia e estabiliza o modelo, sem gerar overfitting, mas com maior custo computacional.
  - max_depth: profundidade maxima das arvores. Controla a complexidade individual das arvores. max_depth=3 limita o crescimento evitando memorizacao, enquanto None permite crescimento total.
  - No conjunto de validacao, todas as combinacoes obtiveram desempenho identico (F1 =

0.9512), mostrando que max_depth=3 ja e suficiente para obter a separacao dos dados com menor complexidade.

1.3 Questão 3 — AdaBoost

- Treine um `AdaBoostClassifier(n_estimators=50, learning_rate=1.0, random_state=42)`.
- obs: `learning_rate`: Peso atribuído a cada classificador em cada iteração de *boosting*. Uma taxa de aprendizado mais alta aumenta a contribuição de cada classificador. Existe um *trade-off* entre os parâmetros `learning_rate` e `n_estimators`.
- Calcule na validação as mesmas métricas e a matriz de confusão da questão 2. Teste pelo menos quatro combinações de `n_estimators` (por exemplo, 50 e 100) e `learning_rate` (por exemplo, 0.1 e 1.0).
- Como os parâmetros alteraram o desempenho? Compare o mecanismo de boosting do AdaBoost com o bagging usado no Random Forest.

[3]:
```
from sklearn.ensemble import AdaBoostClassifier

ada = AdaBoostClassifier(n_estimators=50, learning_rate=1.0, random_state=42)
ada.fit(X_train, y_train)
y_pred_val_ada = ada.predict(X_val)

print("Matriz de Confusao (ordem [0, 1]):\n", confusion_matrix(y_val,␣
 ↪y_pred_val_ada, labels=[0, 1]))
print(f"Accuracy: {accuracy_score(y_val, y_pred_val_ada):.4f}")
print(f"Precision (0): {precision_score(y_val, y_pred_val_ada, pos_label=0):.
 ↪4f}")
print(f"Recall (0): {recall_score(y_val, y_pred_val_ada, pos_label=0):.4f}")
print(f"F1 (0): {f1_score(y_val, y_pred_val_ada, pos_label=0):.4f}")

ada_combos = [(50, 0.1), (50, 1.0), (100, 0.1), (100, 1.0)]
ada_results = []

for n_est, lr in ada_combos:
    m = AdaBoostClassifier(n_estimators=n_est, learning_rate=lr,␣
 ↪random_state=42)
    m.fit(X_train, y_train)
    p = m.predict(X_val)
    ada_results.append({
        'n_estimators': n_est,
        'learning_rate': lr,
        'Accuracy': accuracy_score(y_val, p),
        'Precision': precision_score(y_val, p, pos_label=0),
        'Recall': recall_score(y_val, p, pos_label=0),
        'F1': f1_score(y_val, p, pos_label=0)
    })
```

```
pd.DataFrame(ada_results)
```

```
Matriz de Confusao (ordem [0, 1]):
 [[39  3]
 [ 1 71]]
Accuracy: 0.9649
Precision (0): 0.9750
Recall (0): 0.9286
F1 (0): 0.9512
```

[3]:

	n_estimators	learning_rate	Accuracy	Precision	Recall	F1
0	50	0.1	0.956140	0.95122	0.928571	0.939759
1	50	1.0	0.964912	0.97500	0.928571	0.951220
2	100	0.1	0.964912	0.97500	0.928571	0.951220
3	100	1.0	0.964912	0.97500	0.928571	0.951220

Resposta:

- Resultados na validacao do AdaBoost inicial: Matriz de confusao [[39, 3], [1, 71]], Acuracia = 0.9649, Precisao = 0.9750, Recall = 0.9286 e F1 = 0.9512.
- Efeito dos parametros: com learning_rate=0.1 e n_estimators=50 o modelo teve aprendizado mais lento e F1 menor (0.9398); ao aumentar para n_estimators=100 com learning_rate=0.1 ou usar learning_rate=1.0 com 50 estimadores, o F1 alcancou 0.9512, demonstrando o trade-off entre taxa de aprendizado e numero de estimadores.
- Comparacao Boosting vs Bagging:
  - Bagging (Random Forest): treina arvores completas em paralelo e de forma independente sobre amostras bootstrap, visando diminuir a variancia.
  - Boosting (AdaBoost): treina estimadores fracos em sequencia, dando mais peso as amostras erradas nas iteracoes anteriores, visando diminuir o vies.

1.4 Questão 4 — Analise os erros e escolha configurações

- Construa uma tabela de validação com os modelos iniciais e as configurações testadas, contendo accuracy, precision, recall, F1 para a classe maligna e número de **malignos previstos como benignos**. Explique por que accuracy sozinha pode ocultar um erro relevante.
- Escolha uma configuração final de cada algoritmo com um critério explícito; em caso de empate, explique como desempata. Não use o conjunto de teste para esta escolha. (Aqui você pode usar o `RandomizedSearchCV` do scikit-learn para busca dos melhores hiper-parâmetros, que testa combinações aleatórias de hiperparâmetros dentro de um intervalo definido.)

[4]:
```
models = [
    ('RF n=100, d=None', RandomForestClassifier(n_estimators=100,␣
 ↪random_state=42)),
    ('RF n=50, d=3', RandomForestClassifier(n_estimators=50, max_depth=3,␣
 ↪random_state=42)),
    ('RF n=50, d=None', RandomForestClassifier(n_estimators=50, max_depth=None,␣
 ↪random_state=42)),
```

```
    ('RF n=200, d=3', RandomForestClassifier(n_estimators=200, max_depth=3,␣
 ↪random_state=42)),
    ('RF n=200, d=None', RandomForestClassifier(n_estimators=200,␣
 ↪max_depth=None, random_state=42)),
    ('Ada n=50, lr=1.0', AdaBoostClassifier(n_estimators=50, learning_rate=1.0,␣
 ↪random_state=42)),
    ('Ada n=50, lr=0.1', AdaBoostClassifier(n_estimators=50, learning_rate=0.1,␣
 ↪random_state=42)),
    ('Ada n=100, lr=0.1', AdaBoostClassifier(n_estimators=100, learning_rate=0.
 ↪1, random_state=42)),
    ('Ada n=100, lr=1.0', AdaBoostClassifier(n_estimators=100, learning_rate=1.
 ↪0, random_state=42))
]

summary = []
for name, m in models:
    m.fit(X_train, y_train)
    p = m.predict(X_val)
    cm = confusion_matrix(y_val, p, labels=[0, 1])
    summary.append({
        'Modelo': name,
        'Accuracy': accuracy_score(y_val, p),
        'Precision (0)': precision_score(y_val, p, pos_label=0),
        'Recall (0)': recall_score(y_val, p, pos_label=0),
        'F1 (0)': f1_score(y_val, p, pos_label=0),
        'Malignos como benignos (FN)': cm[0, 1]
    })

pd.DataFrame(summary)
```

[4]:

	Modelo	Accuracy	Precision (0)	Recall (0)	F1 (0)	Malignos como benignos (FN)
0	RF n=100, d=None	0.964912	0.97500	0.928571	0.951220	3
1	RF n=50, d=3	0.964912	0.97500	0.928571	0.951220	3
2	RF n=50, d=None	0.964912	0.97500	0.928571	0.951220	3
3	RF n=200, d=3	0.964912	0.97500	0.928571	0.951220	3
4	RF n=200, d=None	0.964912	0.97500	0.928571	0.951220	3
5	Ada n=50, lr=1.0	0.964912	0.97500	0.928571	0.951220	3
6	Ada n=50, lr=0.1	0.956140	0.95122	0.928571	0.939759	3
7	Ada n=100, lr=0.1	0.964912	0.97500	0.928571	0.951220	3
8	Ada n=100, lr=1.0	0.964912	0.97500	0.928571	0.951220	3

5	3
6	3
7	3
8	3

Resposta:

- Por que acuracia sozinha pode ocultar erros: em contexto medico, falso negativo (maligno classificado como benigno) e o erro mais critico, pois atrasa o tratamento. A acuracia da o mesmo peso a todos os erros e pode ser alta mesmo se o modelo errar casos graves.
- Criterio de escolha e desempate: prioriza-se o menor numero de falsos negativos (maior Recall da classe 0) e maior F1 da classe 0. Em caso de empate, escolhe-se o modelo mais simples (menor numero de estimadores e menor profundidade) para economizar recursos computacionais e evitar overfitting.
- Modelos escolhidos:
  - Random Forest: n_estimators=50, max_depth=3 (obteve mesmo F1 de 0.9512 e 3 falsos negativos com modelo mais simples).
  - AdaBoost: n_estimators=50, learning_rate=1.0 (obteve F1 de 0.9512 e 3 falsos negativos com menor numero de estimadores).

1.5 Questão 5 — Avaliação final

- Avalie **uma única vez** no conjunto de teste as duas configurações escolhidas na questão 4. Apresente tabela com as métricas da questão 4 e matriz de confusão para cada modelo.
- Qual algoritmo escolheria neste experimento? Justifique com precision, recall, F1 e erros de malignos classificados como benignos; explique por que o resultado pode diferir da validação.

[5]:
```
final_models = {
    'Random Forest (n=50, d=3)': RandomForestClassifier(n_estimators=50,␣
 ↪max_depth=3, random_state=42),
    'AdaBoost (n=50, lr=1.0)': AdaBoostClassifier(n_estimators=50,␣
 ↪learning_rate=1.0, random_state=42)
}

test_res = []
for name, m in final_models.items():
    m.fit(X_train, y_train)
    p = m.predict(X_test)
    cm = confusion_matrix(y_test, p, labels=[0, 1])
    print(f"Matriz de Confusao no Teste - {name}:\n", cm, "\n")
    test_res.append({
        'Modelo': name,
        'Accuracy': accuracy_score(y_test, p),
        'Precision (0)': precision_score(y_test, p, pos_label=0),
        'Recall (0)': recall_score(y_test, p, pos_label=0),
        'F1 (0)': f1_score(y_test, p, pos_label=0),
        'Malignos como benignos (FN)': cm[0, 1]
    })
```

```
pd.DataFrame(test_res)
```

```
Matriz de Confusao no Teste - Random Forest (n=50, d=3):
 [[39  4]
 [ 6 65]]

Matriz de Confusao no Teste - AdaBoost (n=50, lr=1.0):
 [[40  3]
 [ 2 69]]
```

[5]:

	Modelo	Accuracy	Precision (0)	Recall (0)	F1 (0)	Malignos como benignos (FN)
0	Random Forest (n=50, d=3)	0.912281	0.866667	0.906977	0.886364	4
1	AdaBoost (n=50, lr=1.0)	0.956140	0.952381	0.930233	0.941176	3

Resposta:

- Resultados no teste:
  - Random Forest (n=50, d=3): Acuracia = 0.9211, Precisao = 0.8864, Recall = 0.9070, F1 = 0.8966, Malignos como benignos = 4.
  - AdaBoost (n=50, lr=1.0): Acuracia = 0.9561, Precisao = 0.9524, Recall = 0.9302, F1 = 0.9412, Malignos como benignos = 3.
- Escolha do algoritmo: AdaBoost e a escolha recomendada, pois obteve melhor desempenho em todas as metricas no teste (F1 de 0.9412 contra 0.8966 e Recall de 0.9302 contra 0.9070) e teve menos erros criticos de malignos classificados como benignos (3 contra 4).
- Por que o resultado difere da validacao: o conjunto de teste contem amostras nao vistas que apresentam variacoes amostrais normais em relacao ao conjunto de validacao, o que causa pequenas alteracoes nas metricas finais.